

OBIETTIVO: trasformare OS/161 in un sistema capace di mantenere correttamente lo stato di più processi contemporaneamente, anche mentre questi entrano contemporaneamente nel kernel. La sincronizzazione viene applicata alle singole strutture condivise.

PUNTO DI INGRESSO DELLE SYSTEM CALL: kern/arch/mips/syscall/syscall.c

Quando un programma user esegue ad es: `read(fd, buffer, 100);` → genera una trap, si entra in kernel mode e `syscall()` che è il dispatcher guarda il numero della syscall contenuto nel trapframe: `read(fd, buffer, 100);`. Poi attraverso lo switch indirizza l'esecuzione verso:

```
sys_read(...)
sys_write(...)
sys_open(...)
sys_fork(...)
sys_execv(...)
sys_waitpid(...)
...
```

CONVENZIONE DI RITORNO:

```
tf->tf_v0 = retval;
tf->tf_a3 = 0;      /* successo */
```

```
tf->tf_v0 = err;
tf->tf_a3 = 1;      /* errore */
```

Dispatcher gestisce l'API e l'architettura MIPS, invece le `sys...` implementano la semantica delle system calls.

SYSTEM CALL DEI FILE:

file_syscalls.c (kern/syscall/file_syscalls.c)

Parliamo di: `open`, `read`, `write`, `lseek`, `close`, `dup2`, `chdir`, `__getcwd`.

Il file “`file_syscalls.c`” si appoggia ad una nuova struttura: `kern/include/openfile.h`

Il nostro problema era: «Dove memorizzo lo stato di un file aperto, considerando che più descrittori e più processi possono dividerlo?»

Quindi la struct `proc` contiene la TABELLA DEI DESCRITTORI DEL SINGOLO PROCESSO (`p_filetable`), invece `openfile` contiene lo STATO DELLA SINGOLA APERTURA DEL FILE.

```
struct openfile {
    struct vnode *of_vnode;
    off_t of_offset;
    int of_flags;
    unsigned int of_refcount;
    struct lock *of_lock;
};
```

Non mettiamo tutto direttamente nella struct proc perché dup2() e fork() devono poter produrre più riferimenti alla stessa apertura.

Quando apro un file ho bisogno di INFO RELATIVE A QUELLA SPECIFICA APERTURA (in che modalità è stato aperto, a che posizione sono arrivato, quanti stanno usando questa apertura), per questo si usa la struct openfile.

FLUSSO: Supponiamo che il processo faccia:

fd = open("prova.txt", O_RDONLY); → fd è un indice che il processo usa per riferirsi al file, un INDICE nella FILE TABLE DEL PROCESSO.

Quando ad es facciamo: read(3, buffer, 100);, il kernel usa 3 per fare:

curproc->p_filetable[3] e **ottenere l'openfile (rappresenta una SPECIFICA APERTURA DEL FILE)**. VNODE invece è l'oggetto nel filesystem che ci permette di fare lettura/scrittura ... con VOP_READ(), VOP_WRITE()...

Il vnode ci dice quale file stiamo usando ma NON contiene lo STATO della specifica APERTURA, per questo non si può fare direttamente: fd→vnode, o meglio p_filetable[3] → vnode.

Rappresenta una specifica apertura di un file nel kernel.

- struct vnode *of_vnode; → indica quale oggetto del VFS è stato aperto. Openfile non contiene direttamente il file su disco, ma contiene un riferimento al vnode e attraverso questo possiamo chiamare VOP_READ, VOP_WRITE...
- **off_t of_offset** → (CONDIVISO) memorizza la posizione corrente. Non va messa nella file descriptor table perché deve essere condiviso in casi come : dup2() o fork(). Per esempio dopo la fork() sia PADRE, che FIGLIO (che hanno stesso fd) devono avere lo stesso offset.
- Of_flags → flag per sapere come è stato aperto file, se O_RDONLY...
- Of_refcount → Più riferimenti puntano allo stesso openfile. Solo quando refcount == 0 possiamo realmente distruggere l'openfile.
- **Of_lock** → serve perché l'offset è condiviso e vogliamo evitare race condition. Protegge of_offset. Serve perché due thread/processi possono leggere o scrivere contemporaneamente sullo stesso openfile.

L'offset è relativo alla specifica apertura (openfile) e non sta nella file table perché con dup2() o fork() diventa condiviso tra più processi.

Poi abbiamo definito:

- OPEN_MAX = max numero di descriptor per processo
- SYSTEM_OPEN_MAX = max numero di openfile per sistema

→ #define SYSTEM_OPEN_MAX (10 * OPEN_MAX) → ho scelto una capacità globale pari a 10 volte il limite per processo, così possiamo avere più processi con più file aperti. E' una scelta progettuale.

FUNZIONI DICHIARATE IN `openfile.h`:

- void `openfile_bootstrap`(void); → inizializzare la struttura globale degli `openfile`.
- int `openfile_open`(const char *path, int flags, int mode, struct `openfile` **result); → incapsula la creazione di un nuovo `openfile`.
- void `openfile_incref`(struct `openfile` *of); → aggiunge un nuovo riferimento
- void `openfile_release`(struct `openfile` *of); → elimina un riferimento

Quando `refcount == 0` chiamiamo `vfs_close()`, viene distrutto il lock e la entry torna libera.

IDEA FONDAMENTALE:

File descriptor → `p_filetable[fd]` → struct `openfile` → `vnode`.

Il `fd` è un indice nella tabella privata del processo. Invece l'`OPENFILE` rappresenta una vera `OPEN FILE DESCRIPTION` = stato associato ad una specifica apertura del file. Il `vnode` invece è l'oggetto `VFS` che rappresenta il file/device sottostante.

Ho scelto di fare un struct `openfile` separata perché alcune info devono essere CONDIVISE tra + file descriptor o più processi. ESEMPIO: Dopo `dup2()` due descriptor devono riferirsi alla **stessa apertura** e quindi condividere anche l'offset. Dopo `fork()`, padre e figlio devono avere file descriptor table separate, ma le entry devono puntare agli stessi `openfile`. Per questo l'offset non può stare dentro `p_filetable[fd]` ma in `openfile`.

VARI LOCK:

```
processTable.lk
    → protegge la tabella globale dei processi

p_filetable_lock
    → protegge i fd di UN processo

system_file_table_lock
    → protegge la tabella globale delle openfile

of_lock
    → protegge lo stato della singola openfile,
    soprattutto l'offset
```

proc.h

(`kern/include/proc.h`)

STRUCT PROC → deve contenere lo STATO PROPRIO DEL PROCESSO.

```
char *p_name;
struct spinlock p_lock;
unsigned p_numthreads;

struct addrspace *p_addrspace;
struct vnode *p_cwd;
```

```

#ifdef OPT_WAITPID
    int p_status;
    pid_t p_pid;
    struct semaphore *p_sem;
#endif

```

- p_pid → identificatore unico del processo.
- P_status → codice di uscita processo
- Semaforo → per la sincronizzazione → niente BUSY WAITING. Serve per la sincronizzazione tra PADRE-FIGLIO con waitpid() ed exit() → il padre fa P() e il figlio quando termina fa V().

```

#ifdef OPT_PDSC2
    pid_t p_parent_pid;

```

Abbiamo introdotto il PID DEL PADRE. Serve per permettere a waitpid() di controllare se il processo che sta facendo waitpid(pid...) è davvero il padre.

Infine abbiamo:

```

struct openfile *p_filetable[OPEN_MAX];
struct lock *p_filetable_lock;

```

P_FILETABLE[OPEN_MAX] = FILE DESCRIPTOR TABLE PRIVATA DEL PROCESSO.

I descriptor 0,1,2 sono stdin, stdout, stderr e sono collegati a **con**:

Abbiamo quindi 2 lock diversi:

- struct lock *p_filetable_lock; (in proc) → protegge la tabella che associa fd → openfile.
- struct lock *of_lock; (in openfile) → protegge stato interno dell'openfile (of_offset)

DIPENDENZA DA RICORDARE: **PDSC2** — richiede **WAITPID** come configurazione.

File_syscalls.c

Qui abbiamo l'architettura principale.

Ci sono 2 livelli:

per-processo:

p_filetable[]

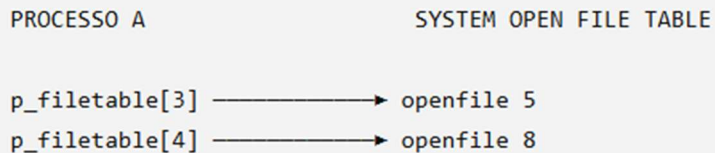
globale:

system_file_table[]

TABELLA GLOBALE: per gestire le open file description del sistema:

```
static struct openfile system_file_table[SYSTEM_OPEN_MAX];  
static struct lock *system_file_table_lock;
```

Questa System Open File table contiene gli openfile disponibili nel sistema.



Si usa una tabella globale perché mi permette di distinguere tra:

- EMFILE-> tabella descritto del singolo processo piena
- ENFILE-> tabella globale degli openfile piena.

FUNZIONI:

- **Openfile_bootstrap()**→ viene chiamata all'avvio del kernel. Inizializza:
 - o System_file_table
 - o System_file_table_lock

Il panic() è accettabile perché se il lock non può essere creato può panicare essendo nel bootstrap e non durante una syscall user.

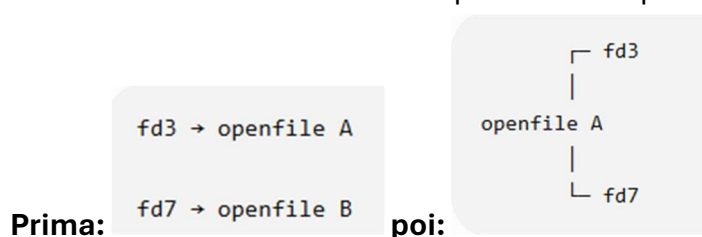
- **Openfile_open()**→ Non è la syscall open(), ma l'helper interno del kernel. Riceve un pathname che è già in kernel space e crea l'openfile.

Questa semplicemente fa: kstrdup(path)→ vfs_open()→ ottengo vnode→ creo of_lock per proteggere of_offset→ cerco slot nella system file table→ inizializzo openfile→ refcount = 1.

Usiamo kstrdup() perché: vfs_open() può modificare il buffer contenente il pathname. Passare direttamente una stringa non modificabile poteva produrre comportamenti strani. Con kstrdup(path) otteniamo una copia privata e scrivibile!.

- **Sys_open()**→ Quando chiamiamo fd = open("file.txt", O_RDONLY);, Il pathname "file.txt" si trova nello **spazio di indirizzamento user**. Il kernel non deve semplicemente fare: strlen(filename); su quel puntatore perché potrebbe essere un indirizzo invalido. Quindi usiamo copyinstr() (copia da user space a kernel space). Poi controlla i flag (O_RDONLY, O_WRONLY...), per i flag sconosciuti ritorna EINVAL. Cerca un descriptor libero nella file table del processo e se non trova spazio ritorna EMFILE, altrimenti chiama openfile_open() che crea openfile globale. Se la tabella globale è piena ritorna ENFILE, altrimenti curproc->p_filetable[fd] = of; e restituisce fd allo user program

- **sys_read()** → per prima cosa dobbiamo verificare che l'fd sia valido, trovare il corrispettivo openfile (of = curproc->p_filetable[fd]), prendere il lock (p_filetable_lock) per recuperare in sicurezza il puntatore per incrementare il reference_count (openfile_incref(of);), leggere dal vnode (VOP_READ(of->of_vnode, &u);), trasferire i dati nello user buffer, aggiornare l'offset e restituire quanti byte sono stati letti.
Struct uio → usata per descrivere un'operazione di I/O, contiene info come: dove trasferire i dati (user buffer), quanti byte, a quale offset (of_offset), lettura o scrittura, user space o kernel space.
Uio_resid = contiene quanti byte NON sono stati trasferiti. Quindi restituiamo size - u.uio_resid.
- **sys_write()** → concettualmente è speculare alla sys_read(): user buffer → uio → VOP_WRITE → vnode → filesystem
Usiamo O_APPEND perché ogni singola write deve partire dalla fine corrente, quindi non facciamo offset = EOF una sola volta durante la open(). Prima della write infatti facciamo VOP_STAT() e otteniamo statbuf.st_size e settiamo l'offset: of->of_offset = statbuf.st_size; per poi fare VOP_WRITE().
- **sys_close()** → Quando chiamiamo close(3), il kernel fa: p_filetable[3] → openfile x . Poi elimina l'associazione: p_filetable[3] = NULL; e poi chiama openfile_release(X); che chiude realmente il vnode quando refcount == 0.
- **sys_lseek()** → modifica solo of_offset. Non legge / non scrive. Quando chiamiamo: lseek(fd, pos, SEEK_SET); → abbiamo: new offset = pos.
Per SEEK_CUR abbiamo: new offset = old offset + pos.
Per SEEK_END dobbiamo prima conoscere la dimensione del file, infatti usiamo VOP_STAT() e leggiamo st-st_size così: new offset = file size + pos.
Se il vnode non supporta lseek: VOP_ISSEEKABLE(...) che restituisce ESPIPE.
- **sys_dup2()** → Supponiamo che chiamiamo: dup2(3, 7);. Quindi vogliamo che fd 7 faccia riferimento alla stessa openfile description di fd3.



Quindi prima recuperiamo l'openfile A, poi openfile B, facciamo puntare fd7 ad A e incrementiamo il refcount di A e rilasciamo B. Non copiamo A perché dup2() deve condividere l'offset. Questo mi permette di implementare la redirectione.

- **sys_chdir()** → riceve il pathname user, lo copia per kernel space con copyinstr() e poi delega al VFS con : vfs_chdir(kernel_path);. Perché non è compito della syscall implementare la ricerca delle directory, la risoluzione dei pathname, ecc... è responsabilità del filesystem. Gli errori quindi arrivano da sotto e vengono propagati.

- **Sys__getcwd()** → qual è la current working directory del processo? Costruiamo una uio verso il buffer user e chiamiamo : `vfs_getcwd(&u)`; così che il VFS trasferisce il pathname nel buffer user. Il numero di byte trasferiti è : `buflen - uio_resid`.

PROBLEMA DURANTE I TEST: abbiamo trovato:

`pwd` nella root

→ `emu0`:

ma faccendo `cd /testbin` e poi `pwd` mi dava: `Function not implemented`.

Ho notato che il flusso di chiamate era questo: `sys__getcwd` → `vfs_getcwd` → `VOP_NAMEFILE` → `emufs_namefile`, dove dentro c'era:

`root`

→ `return 0`

qualsiasi altra directory

→ `ENOSYS`

Quindi non è un errore della nostra syscall ma una limitazione base di `emufs`.

DIFFICOLTA' RISCONTRATE:

Gestione della condivisione degli openfile. La difficoltà principale è stata distinguere il file descriptor dalla open file description. `dup2()` e `fork()` richiedono che più descriptor possano riferirsi allo stesso openfile, condividendo `vnode` e `offset`. Questo ha reso necessario introdurre il reference counting e un lock per-openfile.

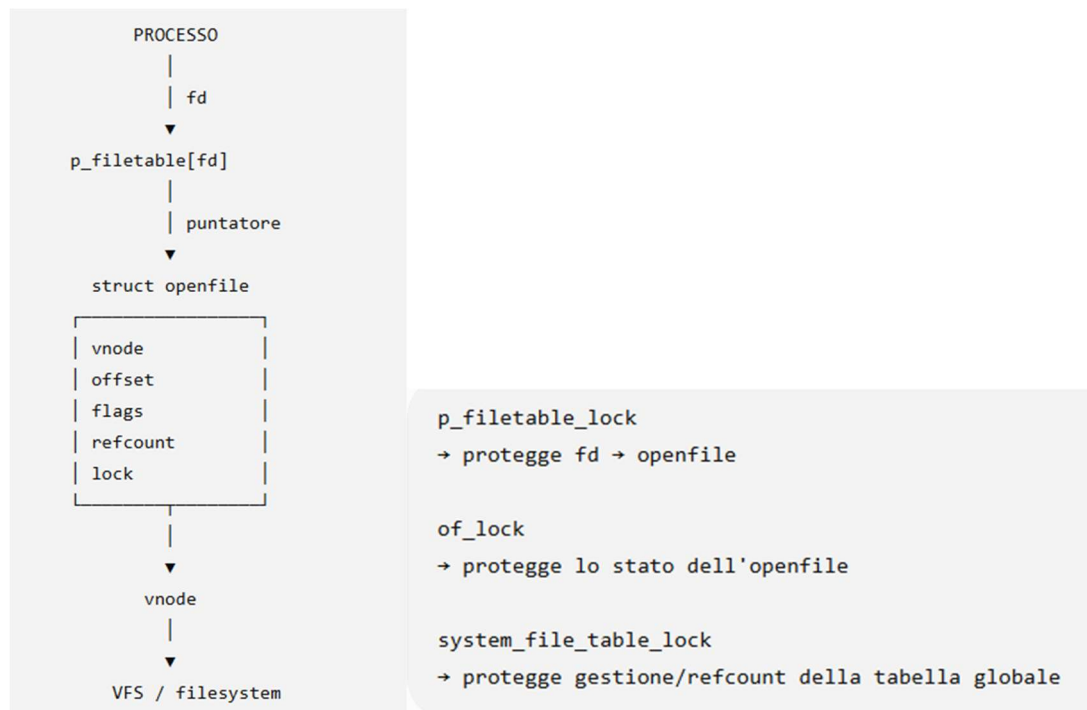
Concorrenza tra `close()` e operazioni in corso. Una volta rilasciato `p_filetable_lock`, un altro thread potrebbe chiudere lo stesso descriptor. Per evitare che l'openfile venga distrutto mentre `read`, `write` o `lseek` lo stanno ancora usando, queste syscall prendono una reference temporanea con `openfile_incref()` e la rilasciano alla fine.

Mutabilità del pathname in `vfs_open()`. Durante l'inizializzazione della console avevamo osservato un comportamento anomalo nelle aperture successive di `con`:. Il problema era legato al fatto che il VFS può modificare il pathname. La soluzione è stata creare una copia scrivibile con `kstrdup()` prima di chiamare `vfs_open()`.

Distinzione tra lock. Non abbiamo usato un unico lock globale per tutto. `p_filetable_lock` protegge la mapping tra descriptor e openfile; `of_lock` protegge lo stato condiviso della singola open file description; `system_file_table_lock` protegge

allocazione e reference counting nella tabella globale. Questo evita sia race condition sia una serializzazione inutile di tutte le operazioni I/O.

__getcwd() e emufs. Il test in una sottodirectory sembrava indicare un errore nella syscall. Seguendo il percorso fino a `emufs_namefile()` abbiamo invece verificato che il filesystem emulato restituisce esplicitamente `ENOSYS` fuori dalla root. Per questo non abbiamo modificato la syscall né esteso `emufs`, che sarebbe stato fuori dallo scope richiesto.



GESTIONE PID, getpid() , fork)

Stiamo adattando OS/161 perché possa **mantenere contemporaneamente più processi, distinguerli, crearli, farli evolvere indipendentemente e poi recuperarne correttamente le risorse.**

Per questa parte i file fondamentali sono:

```
kern/include/proc.h
kern/proc/proc.c
kern/syscall/proc_syscalls.c
kern/arch/mips/syscall/syscall.c
```

Per il ritorno del figlio dopo `fork()`: `kern/arch/mips/syscall/syscall.c`

→ `enter_forked_process()`

```

proc.h
→ definisce quali informazioni deve contenere un processo

proc.c
→ gestisce la vita della struct proc:
    creazione, PID, parentela, thread, distruzione...

proc_syscalls.c
→ implementa il comportamento visibile delle syscall:
    getpid, fork, execv, waitpid, _exit

syscall.c
→ prende gli argomenti dai registri MIPS
    e chiama la sys_* corretta

```

PID → hanno un ciclo di vita. Quindi non basta incrementare sempre: `proc->p_pid = next_pid++`; → se facessimo così prima o poi arriveremmo al limite dei PID e il sistema non potrebbe più creare processi, anche se quelli vecchi sono già terminati.

Quindi dobbiamo risolvere 3 problemi:

- assegnare un PID unico quando nasce un processo
- dato un PID, trovare la relativa struct proc
- quando il processo non serve più, rendere di nuovo disponibile quel PID

Introduciamo **processTable**:

```

struct {
    struct proc *proc[MAX_PROC + 1];
    struct spinlock lk;
} processTable;

```

Usa PID come indice e gli associa in questo array di puntatori a struct proc, la proc

```

PID 1 → processTable.proc[1] → struct proc A
PID 2 → processTable.proc[2] → struct proc B
PID 3 → processTable.proc[3] → struct proc C

```

corrispondente:

Questo è molto utile per syscall come waitpid a cui passiamo solamente il PID del processo da attendere.

Se avessimo soltanto curproc, potremmo conoscere il processo corrente, ma non un processo arbitrario identificato dal PID

Il PID 0 non viene usato come normale PID di processo e ci torna utile anche come valore convenzionale per indicare, per esempio: `p_parent_pid = 0`; (questo processo non ha più un padre valido da attendere). Quindi processTable contiene 101 elementi (`MAX_PROC = 100`).

La processTable è globale quindi può essere usata contemporaneamente da più thread del kernel e per questo motivo deve essere protetta da spinlock (spinlock dato che facciamo operazioni brevi).

- Proc_init_waitpid(): cerchiamo il primo slot libero per assegnare un determinato PID alla struct proc che viene passata:

```
spinlock_acquire(&processTable.lk);

for (pid = 1; pid <= MAX_PROC; pid++) {

    if (processTable.proc[pid] == NULL) {

        processTable.proc[pid] = proc;
        proc->p_pid = pid;
        break;
    }
}

spinlock_release(&processTable.lk);
```

Riutilizziamo gli slot liberi perché la traccia ci chiede di riciclare i PID. La cosa importante è che il riuso avvenga **solo quando il vecchio processo non serve più a nessuno**. Il PID può essere riutilizzato solo quando la relativa struct proc viene effettivamente rimossa e distrutta.

L'ESAURIMENTO dei PID non deve produrre panic, ma deve restituire ENPROC.

- Proc_search_pid(): helper.

```
struct proc *
proc_search_pid(pid_t pid)
{
    struct proc *p;

    if (pid <= 0 || pid > MAX_PROC) {
        return NULL;
    }

    spinlock_acquire(&processTable.lk);

    p = processTable.proc[pid];

    spinlock_release(&processTable.lk);

    return p;
}
```

da PID ottengo direttamente struct proc associata.

- Sys_getpid() → restituisce PID del processo corrente

Non serve un lock per leggere p_pid perché una volta assegnato il PID alla creazione, quel campo non cambia durante la vita del processo.

Fork()

In kern/syscall/proc_syscalls.c:

SYS_FORK:

```
int sys_fork(struct trapframe *ctf,
             pid_t *retval)
{
    struct proc *child;
    struct addrspace *child_as;
    struct trapframe *child_tf;
    int result;
    int create_err;

    KASSERT(ctf != NULL);
    KASSERT(retval != NULL);
    KASSERT(curproc != NULL);

    /*
     * Create the process structure.
     */
    create_err = 0;
    child = proc_create_fork(curproc->p_name, &create_err);
```

```
    if (child == NULL)
    {
        return create_err != 0 ? create_err : ENOMEM;
    }

    /*
     * Copy the parent's address space.
     */
    result = as_copy(curproc->p_addrspace,
                    &child_as);

    if (result)
    {
        proc_destroy(child);
        return result;
    }

    child->p_addrspace = child_as;
```

```
#if OPT_PDSC2
```

```
    /*
     * Copy the file descriptor table.
     *
     * Descriptors remain separate, while openfile structures
     * are shared.
     */
    proc_filetable_copy(curproc, child);
```

```
#endif
```

```
    /*
     * Copy the trapframe.
     */
    child_tf = kmalloc(sizeof(struct trapframe));

    if (child_tf == NULL)
    {
        proc_destroy(child);
        return ENOMEM;
    }

    memcpy(child_tf,
           ctf,
           sizeof(struct trapframe));
```

```

/*
 * Create the child thread.
 */
result = thread_fork(curproc->p_name,
                    child,
                    call_enter_forked_process,
                    child_tf,
                    0);

if (result)
{
    kfree(child_tf);

    proc_destroy(child);

    return result;
}

/*
 * Parent receives the child PID.
 */
*retval = child->p_pid;

return 0;
}

```

fork() deve creare un nuovo processo che inizialmente sia una copia del padre. Dopo la syscall entrambi continuano dall'istruzione successiva a fork, ma:

padre → fork() restituisce PID del figlio

figlio → fork() restituisce 0

La tua implementazione divide volutamente il lavoro tra **creazione del processo, copia dello stato del processo, creazione del thread e preparazione del ritorno in user mode**

Prima creiamo la struct proc del figlio (proc_create_fork() che a sua volta chiama proc_create() e restituisce una nuova struct proc con un nuovo PID e le strutture base del nuovo processo). Fork() non usa la stessa struct proc del padre perché sono 2 processi distinti. Devono avere almeno PID diversi, propria fd table, proprio as, stato di terminazione.

Abbiamo aggiunto errp proc_create() può fallire per diverse cause (se kmalloc fallisce → ENOMEM, altrimenti se nessun PID disponibile ENPROC). Quindi vogliamo sapere perché è fallita la creazione e non avere solo NULL.

Dentro proc_create_fork() :

```

spinlock_acquire(&curproc->p_lock);

if (curproc->p_cwd != NULL) {
    VOP_INCREf(curproc->p_cwd);
    newproc->p_cwd = curproc->p_cwd;
}

spinlock_release(&curproc->p_lock);

```

Il figlio eredita la stessa current working directory del padre, condividono il riferimento allo stesso vnode, non lo copiano e per questo dobbiamo fare `VOP_INCREf(curproc->p_cwd)`.

In struct proc abbiamo aggiunto il campo: `p_parent_pid` → PID del padre:

La scelta del PID evita di conservare direttamente un puntatore alla struct proc del padre, che potrebbe diventare invalido quando il padre termina e viene distrutto.

Così che poi i figli potranno essere trasformati in orfani facendo `p_parentpid = 0`.

Quindi è importante memorizzare chi è il padre: `newproc->p_parent_pid = curproc->p_pid;`

In `sys-fork()` poi copiamo l'address space con `as_copy()` e poi: `newproc->p_parent_pid = curproc->p_pid;`

➔ Il Figlio nasce con la stessa immagine di memoria del padre, anche se sono 2 as distinti.

Non condividiamo direttamente l'address space perché :

opo `fork()` padre e figlio devono poter modificare indipendentemente memoria, stack e variabili. Se entrambi puntassero allo stesso address space, una modifica fatta dal figlio modificherebbe direttamente la memoria del padre. Nel tuo progetto, usando `dumbvm`, fai una **copia effettiva** tramite `as_copy()`. In un sistema più evoluto potresti implementare **copy-on-write**, ma non è quello che fa questa implementazione.

Dopo di che chiamiamo `proc_filetable_copy(curproc, child)` e dentro facciamo:

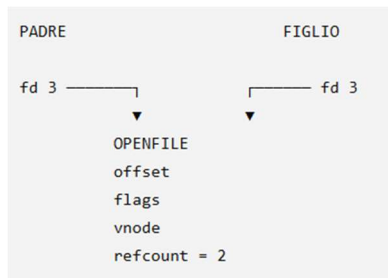
```

dst->p_filetable[fd] = src->p_filetable[fd];
openfile_incref(dst->p_filetable[fd]);

```

quindi copiamo il riferimento ad

`openfile`. Le fd table sono separate, l'`openfile` sottostante è condivisa e quindi anche l'offset è condiviso.



Perché non fai una copia completa della openfile?

Perché padre e figlio devono condividere la stessa **open file description**. Se copiassi la struttura, avresti due offset distinti. Invece dopo `fork()`, se padre e figlio ereditano lo stesso file aperto, gli accessi influenzano lo stesso file offset. Il `refcount` serve invece a evitare che: padre `close(fd)` distrugga la openfile mentre il figlio la sta ancora usando.

Poi copiamo il trapframe: `child_tf = kmalloc(sizeof(struct trapframe));`

`memcpy(child_tf, ctf, sizeof(struct trapframe));`

Questo trapframe contiene lo stato della CPU del PADRE nel momento in cui invochiamo la `fork()`. Il figlio deve riprendere l'esecuzione **esattamente nello stesso punto logico del padre**, cioè immediatamente dopo `fork()`. Con `kmalloc` facciamo una copia indipendente.

Poi con `thread_fork()` creiamo direttamente il thread del figlio da eseguire.

PADRE e FIGLIO devono vedere due valori diversi:

`*retval = child->p_pid; → PADRE vede PID del figlio`

FIGLIO vede 0.

Prima di tornare in user mode dobbiamo fare `as_activate()` per attivare l'as del nuovo processo e poi `mips_usermode(&child_tf)` per passare a user mode.

EXECV

SYS_EXECV()

In `kern/syscall/proc_syscalls.c`

```

fork()
processo padre → crea un SECONDO processo

execv()
processo corrente → rimane lo STESSO processo,
                    ma cambia completamente programma

```

Quindi `execv()` non crea un nuovo PID e non crea una nuova `struct proc`.

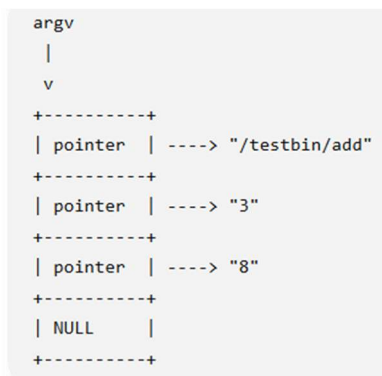
Quello che viene sostituito è principalmente l'address space, cioè il contenuto della memoria virtuale del processo: codice, dati e stack del vecchio programma vengono eliminati e sostituiti con quelli del nuovo eseguibile. In caso di successo `execv()` non ritorna mai al programma che l'ha chiamata, perché l'esecuzione continua direttamente dal punto di ingresso del nuovo programma.

Quali sono i parametri di `sys_execv()`?

```
int sys_execv(userptr_t progname, userptr_t argv)
```

`progname` è un puntatore in spazio utente alla stringa contenente il pathname dell'eseguibile da caricare, per esempio `"/testbin/add"`.

`argv` è invece un puntatore in spazio utente a un array di puntatori a stringhe. Rappresenta quindi gli argomenti da passare al nuovo programma. L'array è terminato da un puntatore NULL



Entrambi i parametri sono `userptr_t`, quindi il kernel non deve mai dereferenziarli direttamente. Deve usare `copyin()`, `copyinstr()`, `copyout()` o `copyoutstr()`, così un indirizzo utente invalido produce EFAULT invece di causare un fault nel kernel.

A cosa servono le principali variabili locali di `sys_execv()`?

- `newas` è il nuovo address space che verrà creato per il programma da eseguire
- `oldas` contiene invece il vecchio address space del processo. Viene conservato temporaneamente perché, se qualcosa fallisce durante l'esecuzione di `execv()`, il kernel può ripristinare il vecchio programma invece di lasciare il processo senza un address space valido.
- `v` è il vnode dell'eseguibile. Viene ottenuto tramite `vfs_open()` e passato a `load_elf()` per leggere e caricare il file ELF.
- `entrypoint` contiene l'indirizzo virtuale della prima istruzione del nuovo programma
- `stackptr` rappresenta il puntatore allo stack del nuovo programma
- `argv_user` memorizza l'indirizzo nello user space in cui viene collocato l'array finale `argv[]`.

- `uargv` è invece un array temporaneo in memoria kernel che contiene gli indirizzi virtuali user delle stringhe già copiate sul nuovo stack.
- `kernel_prog` contiene una copia kernel del pathname del programma
- `kargv` è l'array kernel contenente le copie kernel delle stringhe degli argomenti.
- `arg_buffer` è un buffer temporaneo usato per copiare una stringa alla volta dallo user space al kernel.
- `user_arg` contiene temporaneamente uno dei puntatori presenti nell'array `argv` utente.
- `argc` rappresenta il numero di argomenti, che deve essere calcolato manualmente perché `execv()` riceve solo `argv` e non riceve `argc`.
- `argbytes` tiene traccia della quantità complessiva di memoria occupata dalle stringhe degli argomenti, mentre `actual` contiene la dimensione effettiva della stringa copiata da `copyinstr().len` viene usato successivamente per calcolare la dimensione di ciascuna stringa quando viene inserita nello stack.
-

Perché `programe` viene copiato nel kernel con `copyinstr()` ?

`programe` appartiene allo user space, quindi non è sicuro utilizzarlo direttamente nel kernel. La chiamata a `copyinstr()` copia il pathname in un buffer appartenente al kernel e contemporaneamente verifica che il puntatore utente sia valido. Il kernel lavora quindi con `kernel_prog`.